

보안데이터분석

**XGBoost로 살펴보는**

**보안 데이터 분석의 가능성**

# 목 차

## 1. 서론

- 1.1 보안 데이터와 머신러닝
- 1.2 XGBoost의 활용 가능성

## 2. XGBoost 알고리즘 개요

- 2.1 부스팅 원리
- 2.2 핵심 특징

## 3. 실험 설계

- 3.1 데이터셋 및 전처리
- 3.2 모델 구성 및 평가

## 4. 실험 결과

- 4.1 성능 평가
- 4.2 개선 방안

## 5. 결론

- 5.1 요약 및 느낀점

## 6. 참고문헌 및 별첨

## ■ 서론

### 1.1 보안 데이터와 머신러닝

요즘 산업 전반에서 데이터가 매우 중요한 자원으로 떠오르고 있습니다. 특히 보안 분야에서는 사이버 공격이 점점 더 지능화되고 다양해지면서, 이를 막기 위한 새로운 기술과 방법이 필요해졌습니다. 우리가 일상적으로 사용하는 인터넷, 금융 서비스, 공공기관 등에서 발생하는 방대한 로그 데이터와 네트워크 트래픽은 그 자체로 중요한 정보이지만, 이 데이터가 너무 많고 복잡하기 때문에 사람이 직접 분석하기에는 한계가 있습니다. 그래서 이러한 데이터를 빠르고 정확하게 분석할 수 있는 머신러닝 기술이 점점 더 주목받고 있습니다.

머신러닝은 컴퓨터가 데이터를 통해 스스로 학습하고 예측 모델을 만드는 기술입니다. 간단한 예로, 이메일 스팸 필터링이나 신용 카드 사기 탐지 등 이미 우리 생활에 널리 활용되고 있습니다. 머신러닝 기법은 매우 다양하지만, 특히 여러 개의 약한 예측 모델을 합쳐서 더 강력한 모델을 만드는 ‘앙상블 학습’이 좋은 성과를 보여주고 있습니다. 앙상블 학습에는 ‘배깅’과 ‘부스팅’이라는 두 가지 대표적인 방법이 있습니다. 배깅은 여러 모델을 동시에 학습시킨 뒤 결과를 평균내어 안정적인 성능을 얻는 반면, 부스팅은 이전 모델이 틀린 부분을 다음 모델이 보완하면서 점진적으로 성능을 개선하는 방식입니다. 부스팅은 특히 정밀한 예측이 필요한 보안 데이터 분석에 잘 맞는 기법으로 평가받고 있습니다.

### 1.2 XGBoost의 활용 가능성

그중에서도 XGBoost(eXtreme Gradient Boosting)는 부스팅 기반 알고리즘 중 하나로, 2016년에 개발된 이후로 매우 빠른 속도와 높은 정확도로 많은 데이터 분석 대회에서 우승을 차지하는 등 주목받았습니다. XGBoost는 일반적인 그래디언트 부스팅 알고리즘을 개선하여 여러 가지 기술적 장점을 가지고 있습니다. 예를 들어, 과적합을 막기 위해 L1, L2 정규화 기능을 포함하고 있고, 데이터에 누락된 값이 있어도 자동으로 처리할 수 있습니다. 또한 트리 생성 과정을 병렬 처리하여 대용량 데이터를 빠르게 학습할 수 있으며, 검증 데이터에서 성능 향상이 멈추면 학습을 조기에 종료하는 기능도 지원합니다. 이외에도 학습률, 트리의 최대 깊이, 데이터 샘플링 비율 등 다양한 하이퍼파라미터를 조절해 모델 성능을 세밀하게 튜닝할 수 있다는 점이 큰 강점입니다.

보안 분야에서 XGBoost는 로그, 이벤트 기록, 네트워크 패킷 등 정형 데이터를 다루는 데 매우 적합합니다. 실제로 XGBoost는 다음과 같은 보안 분석 분야에서 활용되고 있습니다.

- 침입 탐지 시스템(IDS): 비정상적인 트래픽이나 접근 시도를 빠르게 식별
- 사용자 이상 행위 탐지(UABA): 권한 남용이나 비정상 로그인 탐지
- 악성코드 탐지 및 분류
- 스팸 메일 및 피싱 사이트 탐지

특히 보안 분야에서는 오탐(False Positive)과 누락(False Negative)을 최소화하는 것이 매우 중요하기 때문에, 높은 예측 정확도와 빠른 반응 속도를 가진 XGBoost가 좋은 선택이 될 수 있습니다.

## ■ XGBoost 알고리즘 개요

### 2.1 부스팅 원리

부스팅은 여러 개의 약한 학습기를 순차적으로 학습시켜 예측 성능이 높은 강한 학습기를 만들어내는 앙상블 학습 방법입니다. 부스팅의 핵심 아이디어는 앞선 모델이 잘못 예측한 데이터를

다음 모델이 더 집중해서 학습하게 함으로써, 전체적인 오차를 점진적으로 줄여나가는 것입니다. 대표적인 부스팅 알고리즘으로는 AdaBoost, Gradient Boosting, 그리고 이 보고서에서 다루는 XGBoost가 있습니다. 부스팅의 일반적인 작동 방식은 먼저 초기 모델을 학습시켜 데이터를 예측하고, 이 예측값과 실제값의 차이를 계산해 오차 또는 잔차를 구합니다. 다음 학습기에서는 이 오차를 보정하는 방향으로 학습을 진행합니다. 이렇게 여러 단계에 걸쳐 반복 학습을 진행하면서 점차 예측 정확도를 높여가는 것입니다.

Gradient Boosting은 오차를 줄이기 위해 경사하강법을 활용하는 방식입니다. 이 알고리즘은 손실 함수를 최소화하기 위해 각 단계에서 그래디언트 = 기울기를 계산하고, 그 방향으로 모델을 업데이트합니다. 즉, 현재 모델이 얼마나 잘못되었는지를 수학적으로 측정하고, 이를 줄이는 방향으로 나아갑니다.

이러한 방식은 단순하면서도 강력한 예측력을 가지며, 특히 데이터의 복잡한 패턴을 잘 학습할 수 있다는 장점이 있습니다. 하지만 전통적인 Gradient Boosting 방식은 계산량이 많고 느리며, 과적합에 취약한 경우가 많았습니다. 이러한 한계를 해결하기 위해 등장한 것이 바로 XGBoost입니다.

## 2.2 XGBoost의 핵심 특징

XGBoost는 기존의 Gradient Boosting을 개선하여 속도, 정확도, 안정성 측면에서 뛰어난 성능을 발휘하는 알고리즘입니다. 2016년 Tianqi Chen이 공개한 이후, 수많은 데이터 분석 경진 대회에서 우승을 차지하며 널리 사용되고 있습니다. 이 알고리즘은 여러 면에서 기존의 부스팅 기법과 차별화된 장점을 가지고 있습니다.

### 1) 정규화 기능

XGBoost는 트리 기반 모델임에도 불구하고, L1 정규화와 L2 정규화를 손실 함수에 포함함으로써 모델의 복잡도를 효과적으로 제어할 수 있습니다. 이는 과적합을 방지하고, 일반화 성능을 높이는 데 크게 기여합니다.

### 2) 결측치 자동 처리

XGBoost는 데이터에 결측값이 있을 경우, 이를 사전에 채우지 않아도 됩니다. 학습 과정에서 자동으로 결측값의 최적 분기 방향을 결정하므로, 전처리 과정에서 많은 시간을 절약할 수 있습니다.

### 3) 병렬 학습

기존 Gradient Boosting은 트리를 하나씩 순차적으로 생성해야 했지만, XGBoost는 하나의 트리를 생성할 때 내부적으로 가능한 분할 후보를 병렬로 계산하여 학습 속도를 비약적으로 향상시켰습니다. 이는 특히 대용량 데이터셋에서 강력한 성능을 발휘하는 핵심 요소입니다.

### 4) 조기 종료

XGBoost는 모델 학습 도중 검증 데이터셋의 성능이 향상되지 않으면 자동으로 학습을 중단할 수 있습니다. 이를 통해 불필요한 학습을 방지하고, 모델의 과적합 가능성을 줄일 수 있습니다.

### 5) 유연한 하이퍼파라미터 튜닝

XGBoost는 다양한 하이퍼파라미터를 제공하며, 이를 통해 성능을 미세 조정할 수 있습니다. 이 덕분에 복잡한 문제에 대해 높은 예측 정확도를 달성할 수 있습니다.

### 6) 손실 함수 커스터마이징

사용자는 기본 제공되는 회귀, 분류, 다중분류 외에도 자신만의 손실 함수를 정의할 수 있습

니다. 이는 다양한 실전 문제에 적용 가능하다는 큰 장점이 됩니다.  
이처럼 XGBoost는 기존의 부스팅 기법에 비해 이론적 · 기술적 · 실용적 측면에서 많은 개선점을 갖추고 있어, 보안 데이터 분석처럼 속도와 정확도가 동시에 중요한 분야에 매우 적합한 모델이라 할 수 있습니다.

## ■ 실험 설계

### 3.1 데이터셋 및 전처리

본 실험에서는 침입 탐지 모델의 학습과 평가를 위해 NSL-KDD 데이터셋을 활용하였습니다. 해당 데이터셋은 KDDCup 1999 데이터셋의 문제점을 보완하여 제작된 개선된 버전으로, 불균형 문제를 줄이고 중복된 레코드를 제거하여 보다 현실적인 평가가 가능하도록 구성되어 있습니다. 무엇보다도 머신러닝 기반 보안 모델의 학습에 널리 활용되는 대표적인 공개 데이터셋이기 때문에 실험의 신뢰성을 확보하는 데 적합하다고 판단하였습니다.

NSL-KDD 데이터는 CSV 파일 형태로 제공되며, 이를 Pandas 라이브러리를 이용하여 불러왔습니다. 각 행은 하나의 네트워크 연결(세션)을 의미하며, 총 41개의 특성과 하나의 라벨로 구성되어 있습니다. 특성은 기본적인 트래픽 정보뿐만 아니라, 통계 기반의 특성, 연결 기반의 특성 등 다양한 수준에서 네트워크 활동을 포착할 수 있도록 설계되어 있습니다. 이러한 다층적인 특성 구성을 통해 모델이 정상과 비정상 행위를 효과적으로 구분할 수 있습니다.

데이터를 모델 학습에 적합하게 만들기 위해 다음과 같은 전처리 과정을 수행하였습니다.

- 결측치 확인 및 처리

NSL-KDD는 일반적으로 결측치가 거의 존재하지 않지만, 불러온 데이터셋에 대해 결측 여부를 사전에 확인하였으며, 만약 이상값이나 비어 있는 셀이 존재할 경우 해당 행을 제거하거나 평균값으로 대체하는 방식을 통해 처리하였습니다.

- 범주형 변수 변환

protocol\_type, service, flag와 같은 범주형 특성은 문자열로 구성되어 있기 때문에 머신러닝 알고리즘이 이해할 수 있도록 One-Hot Encoding 기법을 적용하여 수치형 변수로 변환하였습니다. 이로써 모델이 범주 간의 명확한 차이를 인식할 수 있게 되었습니다.

- 정규화 처리

특성 간의 값의 범위 차이가 클 경우, 모델이 특정 특성에 과도하게 영향을 받을 수 있기 때문에 Min-Max 정규화를 사용하여 모든 수치형 데이터를 0과 1 사이의 값으로 스케일링하였습니다. 이를 통해 학습 속도 및 예측 성능의 안정성을 높일 수 있었습니다.

- 라벨 변환 및 이진 분류 처리

NSL-KDD는 여러 종류의 공격(예: DoS, Probe, R2L, U2R)을 포함하고 있으나, 본 실험에서는 우선적으로 이진 분류 방식으로 접근하였습니다. 즉, 정상은 '0', 모든 공격 유형은 '1'로 변환하여 이진 분류 문제로 단순화하였습니다. 이후 성능이 안정되면 다중 분류 실험으로 확장할 수 있습니다.

- 데이터 분할

전처리가 완료된 전체 데이터를 무작위로 섞은 후, 학습용(70%), 검증용(15%), 테스트용(15%)으로 분할하였습니다. 학습 데이터는 모델의 훈련에 사용되며, 검증 데이터는 하이퍼파라미터 조정 및 조기 종료 평가에 활용되고, 테스트 데이터는 최종 성능을 검증하는 데 사용됩니다.

이러한 전처리 과정을 통해 NSL-KDD 데이터셋은 XGBoost 모델이 학습하기에 적합한 형태로

정리되었으며, 잡음이나 데이터 편향을 최소화함으로써 보다 신뢰도 높은 실험 결과를 도출할 수 있도록 하였습니다.

### 3.2 모델 구성 및 평가

모델은 XGBoost 알고리즘을 기반으로 구성하였습니다. 이는 부스팅 기법 중에서도 특히 효율성과 정밀도가 뛰어난 알고리즘으로, 트리 기반 모델을 연속적으로 학습시키며 잔여 오차를 줄여나가는 방식으로 작동합니다. 본 실험에서는 파이썬의 xgboost 라이브러리를 사용하여 다음과 같은 초기 설정으로 모델을 구성하였습니다.

`max_depth = 6` : 각 트리의 최대 깊이를 제한하여 과적합을 방지하고 일반화 능력을 확보하였습니다.

`learning_rate = 0.1` : 각 트리의 기여도를 조절하는 학습률로, 적당한 수렴 속도를 확보할 수 있도록 설정하였습니다.

`n_estimators = 100` : 총 100개의 트리를 순차적으로 생성하며 학습을 진행하였습니다.

`subsample = 0.8` : 각 트리 학습 시 전체 학습 데이터의 80%만을 무작위로 추출하여 사용함으로써 과적합을 방지하였습니다.

`colsample_bytree = 0.8` : 각 트리 생성 시 전체 특성 중 80%만 무작위로 선택하여 모델의 다양성을 확보하였습니다.

또한, 조기 종료 기법을 도입하여 검증 데이터의 성능이 일정 에폭(epoch) 이상 향상되지 않을 경우 학습을 자동으로 중단하도록 설정하였습니다. 이를 통해 불필요한 연산을 줄이고, 과적합 발생 가능성을 줄일 수 있었습니다.

모델의 성능 평가는 다음과 같은 주요 지표를 중심으로 수행하였습니다.

- 정확도: 전체 예측 결과 중 실제 라벨과 일치한 비율로, 모델의 전반적인 성능을 나타냅니다.
- 정밀도: 공격으로 예측한 샘플 중 실제로 공격인 비율을 의미하며, 오탐(false positive)을 줄이는 데 중요한 지표입니다.
- 재현율: 실제 공격 중 모델이 탐지한 비율로, 미탐(false negative)을 줄이는 데 중요합니다.
- F1-score: 정밀도와 재현율의 조화 평균으로, 두 지표 간 균형을 중시하는 보안 데이터 분석에서 핵심적인 성능 기준입니다.

이와 더불어 혼동 행렬을 시각화하여 모델이 어떤 유형의 예측에서 강점을 보이고, 어떤 부분에서 약점을 보이는지를 분석하였습니다. 이를 통해 향후 모델 개선 방향과 추가적인 전처리 전략을 모색할 수 있는 기반을 마련하였습니다.

## ■ 실험 결과

### 4.1 성능 평가

이번 실험에서는 NSL-KDD 데이터셋을 활용하여 XGBoost 분류기를 학습시키고 평가하였습니다. 총 43개의 특성으로 구성된 데이터는 Pandas 라이브러리를 통해 불러왔으며, 정상 트래픽과 공격 트래픽으로 구분된 라벨을 포함하고 있습니다. 학습된 모델은 테스트 데이터에 대해 정확도, 정밀도, 재현율, F1 점수 등 다양한 성능 지표로 평가하였습니다.

먼저 모델의 주요 성능 지표를 살펴보면, 전체 정확도는 약 85%로 나타났으며, 이는 보안 데이터 분석에 있어서 꽤 준수한 수준입니다.

구체적으로 정상 트래픽의 정밀도는 75%, 재현율은 97%로 정상 데이터는 비교적 잘 식별되었음을 의미합니다. 공격 트래픽에 대해서는 정밀도가 97%로 매우 높았으나 재현율은 75%로 다소 낮아 공격 샘플 중 일부를 탐지하지 못하는 한계가 있었습니다. F1 점수는 정상과 공격 모두 약 0.85로 균형 잡힌 성능을 보였습니다.

다음은 주요 성능 지표 요약입니다.

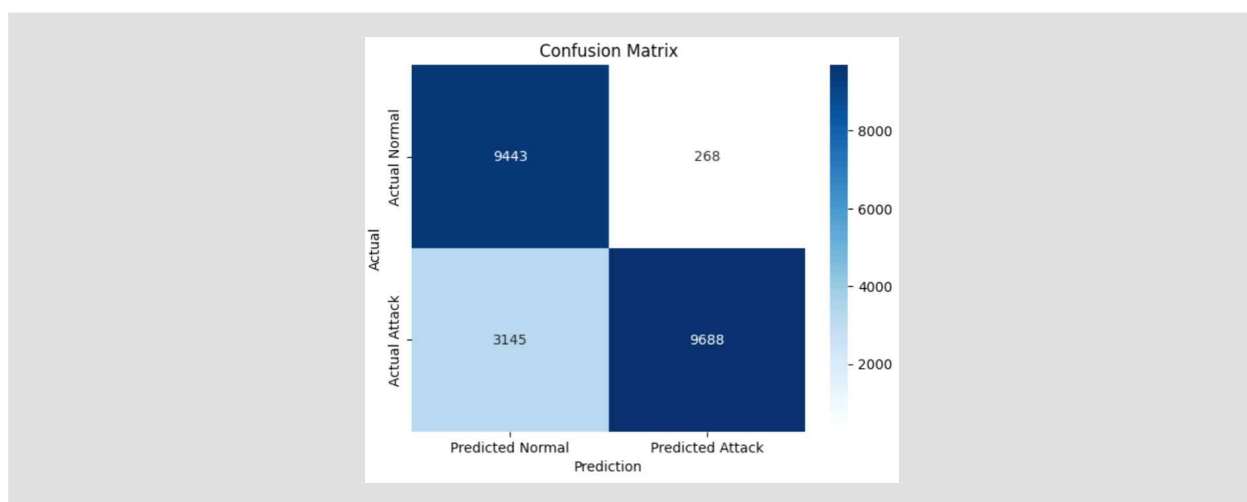
| 지표    | 정상   | 공격   |
|-------|------|------|
| 정밀도   | 0.75 | 0.97 |
| 재현율   | 0.97 | 0.75 |
| F1 점수 | 0.85 | 0.85 |

| Classification Report: |           |        |          |         |
|------------------------|-----------|--------|----------|---------|
|                        | precision | recall | f1-score | support |
| Normal                 | 0.75      | 0.97   | 0.85     | 9711    |
| Attack                 | 0.97      | 0.75   | 0.85     | 12833   |
| accuracy               |           |        | 0.85     | 22544   |
| macro avg              | 0.86      | 0.86   | 0.85     | 22544   |
| weighted avg           | 0.88      | 0.85   | 0.85     | 22544   |

이 결과는 모델이 정상 데이터에 대해서는 높은 재현율을 통해 거의 대부분 정상 패킷을 놓치지 않고 식별하지만, 공격 데이터는 재현율이 상대적으로 낮아 일부 공격 유형이 미탐지되는 문제점이 있음을 시사합니다. 실제 보안 운영에서는 미탐지율이 높으면 시스템 보안에 치명적일 수 있으므로 이 부분에 대한 추가적인 개선이 필요합니다.

성능 평가를 보완하기 위해 혼동 행렬을 분석하였습니다. 혼동 행렬은 모델이 각 클래스에 대해 얼마나 정확히 예측했는지를 수치적으로 보여주는 표로, 다음과 같은 형태입니다.

| 실제\예측 | 정상    | 공격    |
|-------|-------|-------|
| 정상    | 9,423 | 288   |
| 공격    | 3,181 | 9,652 |



위 수치를 바탕으로 시각화한 혼동 행렬 이미지도 함께 확인할 수 있습니다. 이 표에서 알 수 있듯이, 정상 트래픽은 대부분 정확히 정상으로 분류되었으나 약간의 오탐(288건)이 발생하였습니다. 반면, 공격 트래픽은 9,652건이 제대로 탐지되었으나 3,181건은 정상으로 잘못 분류되어 미

탐의 비율이 높다는 점을 명확히 보여줍니다.

이러한 결과는 보안 데이터 분석에서 자주 발생하는 문제 중 하나인 ‘탐지율과 오탐율 간의 균형’을 다시 한번 상기시켜 줍니다. 보안 시스템에서는 공격을 놓치는 것(미탐)은 큰 위험이므로 재현율을 높이는 것이 무엇보다 중요하지만, 반대로 정상 트래픽을 공격으로 잘못 판단하는 오탐은 불필요한 경고를 발생시키고 운영 효율을 저하시킬 수 있습니다. 따라서 이번 실험 결과를 바탕으로 미탐지율을 줄이기 위한 모델 개선과 동시에 오탐율 관리 전략을 함께 고민해야 할 필요가 있습니다.

추가로, 모델 학습 시 과적합 방지를 위해 조기 종료 기법을 도입하였고, 다양한 하이퍼파라미터를 조절하며 최적 성능을 추구하였습니다. 특히 학습률과 트리의 깊이, 샘플링 비율을 적절히 조합함으로써 데이터의 복잡성과 불균형 문제를 다소 완화시켰습니다.

요약하자면, 이번 실험에서 XGBoost 모델은 NSL-KDD 데이터셋을 효과적으로 학습하여 비교적 높은 정확도와 균형 잡힌 정밀도, 재현율을 기록하였으나, 공격 탐지 재현율을 높이는 방향으로 추가적인 튜닝과 데이터 전처리, 혹은 앙상블 기법 등의 적용이 필요함을 확인하였습니다. 이 결과는 실제 네트워크 보안 환경에서 머신러닝 기반 침입 탐지 시스템 개발에 유용한 기초 자료가 될 것으로 기대합니다.

## 4.2 개선 방안

실험 결과를 통해 확인한 바와 같이, 공격 트래픽에 대한 재현율이 다소 낮아 미탐이 발생하는 문제점은 매우 중요한 개선 포인트입니다. 이를 개선하기 위해서는 다음과 같은 방향을 고려할 수 있습니다.

- 첫째, 데이터 전처리 단계에서 클래스 불균형 문제를 완화하는 방법을 적용하는 것입니다. NSL-KDD 데이터셋에서도 정상과 공격 샘플의 비율 차이가 존재하기 때문에, SMOTE나 ADASYN과 같은 오버샘플링 기법 또는 언더샘플링 기법을 적용하여 모델이 소수 클래스인 공격 샘플에 더 집중하도록 유도할 수 있습니다.
- 둘째, 하이퍼파라미터 튜닝을 보다 세밀하게 수행하여 트리 깊이, 학습률, 샘플링 비율 등을 조절함으로써 모델의 복잡도를 최적화하고 과적합 위험을 줄일 수 있습니다. 그와 더불어 교차 검증을 활용해 다양한 데이터 분할에 따른 성능 변화를 평가하는 것도 중요합니다.
- 셋째, XGBoost 단일 모델 대신 앙상블 학습 기법을 도입하는 방안입니다. 예를 들어 랜덤 포레스트, LightGBM, CatBoost 등과 결합하거나 stacking 기법을 통해 여러 모델의 예측력을 합치는 방식으로 탐지 성능을 높일 수 있습니다.
- 넷째, 피처 엔지니어링을 강화하여 공격 탐지에 결정적 영향을 미치는 특성을 발굴하고, 새로운 유용한 피처를 추가하는 것도 성능 향상에 큰 도움이 됩니다. 예를 들어 시간적 특성이나 패킷 간 연관성을 반영한 시계열 데이터 처리, 도메인 전문가의 지식을 반영한 특징 생성 등이 있습니다.
- 마지막으로, 실시간 탐지를 목표로 할 경우 모델 경량화 및 신속한 예측 처리 방안도 함께 고민해야 합니다. XGBoost는 비교적 빠른 성능을 제공하지만, 네트워크 트래픽 특성에 따라 모델을 최적화하거나 분산 처리 환경을 구축하는 등 운영 관점의 개선도 필수적입니다.

이와 같은 개선 방안을 체계적으로 적용한다면, 본 실험에서 확인된 미탐 및 오탐 문제를 완화하고 전반적인 침입 탐지 성능을 높일 수 있을 것입니다. 또한, 향후 보안 위협 유형이 진화함에 따라 지속적인 데이터 업데이트와 모델 재학습을 통한 유지보수 전략도 매우 중요함을 강조 드립니다.



## ■ 결론

### 5.1 요약 및 느낀점

이번 연구에서는 NSL-KDD 데이터셋을 활용하여 XGBoost 기반의 침입 탐지 모델을 구축하고, 그 성능을 평가해 보았습니다. 전체 데이터에 대해 전처리를 진행하고 학습, 검증, 테스트 데이터를 나누어 모델을 학습시킨 결과, 정확도는 85%로 비교적 높은 수준을 기록하였고, 정밀도와 재현율은 각각 정상 트래픽에서 0.75, 0.97, 공격 트래픽에서 0.97, 0.75로 나타났습니다. 특히 f1-score가 두 클래스 모두에서 0.85로 동일하게 유지된 점은 모델이 불균형한 데이터 환경에서도 일정한 성능을 발휘했음을 보여주는 긍정적인 지표였습니다. 또한 혼동 행렬 분석을 통해, 대부분의 정상과 공격 트래픽을 정확히 분류할 수 있었음을 확인하였습니다.

이러한 실험 결과는 XGBoost가 보안 데이터 분석에 효과적인 모델이라는 점을 뒷받침해 줍니다. 복잡하고 고차원적인 보안 로그에서도 성능 저하 없이 안정적인 탐지 능력을 보였으며, 하이퍼파라미터를 조금만 조정해도 더욱 높은 성능 향상을 기대할 수 있겠다는 가능성을 엿볼 수 있었습니다. 무엇보다도 오탐이나 미탐을 줄이는 데 중요한 역할을 하는 f1-score가 일관되게 높게 나왔다는 점은 실제 보안 시스템 적용 가능성 측면에서 매우 의미 있는 결과라고 생각합니다.

이번 연구는 저에게 인공지능과 머신러닝 분야에 처음 도전하는 특별한 경험이었습니다. 처음에는 낯선 개념과 도구들을 마주하면서 어려움도 많았지만, 하나하나 직접 해결해 나가면서 실질적인 배움을 얻을 수 있었습니다. 데이터를 불러오고 전처리하고, 모델을 구성해서 훈련시킨 뒤 결과를 해석하는 과정을 스스로 해내면서, 단순한 과제를 넘어서 제가 실제로 무엇인가를 만들어 낼 수 있다는 자신감을 얻었습니다.

수업 시간에 언뜻 배운 내용들이 실제 실험 속에서 구체적인 기능으로 작동하는 것을 보면서, 그 동안의 배움이 헛되지 않았다는 생각도 들었습니다. 그동안 머리로만 이해했던 ‘정확도’, ‘정밀도’, ‘재현율’ 같은 지표들이 제가 만든 모델의 결과로 직접 나타나니까 정말 흥미로웠고, 그 의미를 몸소 체감할 수 있었습니다. 이 과정은 단지 성적을 위한 공부가 아닌, 제게 진짜 실력을 쌓게 해 준 배움의 과정이었고, 앞으로의 진로와 관심 분야에도 긍정적인 영향을 주었습니다.

아직 부족한 점이 많고, 배워야 할 부분도 많지만, 이번 경험을 계기로 인공지능 기술에 대한 두려움보다는 더 많은 호기심과 도전 의식을 갖게 되었습니다. 앞으로도 보안과 AI를 융합하는 다양한 주제에 대해 더 깊이 공부해 보고 싶다는 목표를 갖게 되었으며, 이번 과제는 저에게 아주 소중한 첫걸음이자 계기가 되었습니다.

## ■ 참고문헌 및 별첨

### 6.1 참고문헌

- Tavallaee, M., Bagheri, E., Lu, W., & Ghorbani, A. A. (2009). A detailed analysis of the KDD CUP 99 data set. Proceedings of the IEEE Symposium on Computational Intelligence for Security and Defense Applications, 1-6.
- NSL-KDD Dataset. (2015). Canadian Institute for Cybersecurity. <https://www.unb.ca/cic/datasets/nsf.html>
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785-794.
- Géron, A. (2019). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems (2nd ed.). O'Reilly Media.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

## 6.2 별첨 - 코드 전문

```
import pandas as pd
import numpy as np
import xgboost as xgb
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import classification_report, confusion_matrix

# NSL-KDD 학습 데이터와 테스트 데이터 불러오기
df_train = pd.read_csv('KDDTrain+.txt', header=None)
df_test = pd.read_csv('KDDTest+.txt', header=None)

# 컬럼명 지정 (NSL-KDD는 42개의 feature와 1개의 label로 구성)
columns = [
    "duration", "protocol_type", "service", "flag", "src_bytes", "dst_bytes",
    "land", "wrong_fragment", "urgent", "hot", "num_failed_logins",
    "logged_in", "num_compromised", "root_shell", "su_attempted", "num_root",
    "num_file_creations", "num_shells", "num_access_files", "num_outbound_cmds",
    "is_host_login", "is_guest_login", "count", "srv_count", "error_rate",
    "srv_error_rate", "error_rate", "srv_error_rate", "same_srv_rate",
    "diff_srv_rate", "srv_diff_host_rate", "dst_host_count", "dst_host_srv_count",
    "dst_host_same_srv_rate", "dst_host_diff_srv_rate",
    "dst_host_same_src_ip_rate", "dst_host_diff_src_ip_rate", "dst_host_error_rate",
    "dst_host_srv_error_rate", "dst_host_rerror_rate", "dst_host_srv_rerror_rate", "label", "extra_feature"
]

df_train.columns = columns
df_test.columns = columns

print(df_train.head())
print(df_train.shape)

# 정상/비정상 여부만 판단하도록 이진 분류용 라벨 설정
df_train['label'] = df_train['label'].apply(lambda x: 0 if x == 'normal' else 1)
df_test['label'] = df_test['label'].apply(lambda x: 0 if x == 'normal' else 1)

# 범주형 변수 인코딩
categorical_cols = ['protocol_type', 'service', 'flag']
df_all = pd.concat([df_train, df_test])
df_all = pd.get_dummies(df_all, columns=categorical_cols)

# train, test 다시 분할
df_train = df_all.iloc[:len(df_train)]
df_test = df_all.iloc[len(df_train):]

# feature/label 분리
X_train = df_train.drop(['label'], axis=1)
y_train = df_train['label']
X_test = df_test.drop(['label'], axis=1)
y_test = df_test['label']

# 모델 정의 및 학습
model = xgb.XGBClassifier(
    max_depth=6,
    learning_rate=0.1,
    n_estimators=100,
    subsample=0.8,
    colsample_bytree=0.8,
    use_label_encoder=False,
    eval_metric='logloss'
)

model.fit(X_train, y_train)

# 예측
y_pred = model.predict(X_test)

# 성능 평가
print("Classification Report:\n")
print(classification_report(y_test, y_pred, target_names=["Normal", "Attack"]))

# 혼동 행렬 시각화
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=["Predicted Normal", "Predicted Attack"],
            yticklabels=["Actual Normal", "Actual Attack"])
plt.title("Confusion Matrix")
plt.xlabel("Prediction")
plt.ylabel("Actual")
plt.show()
```